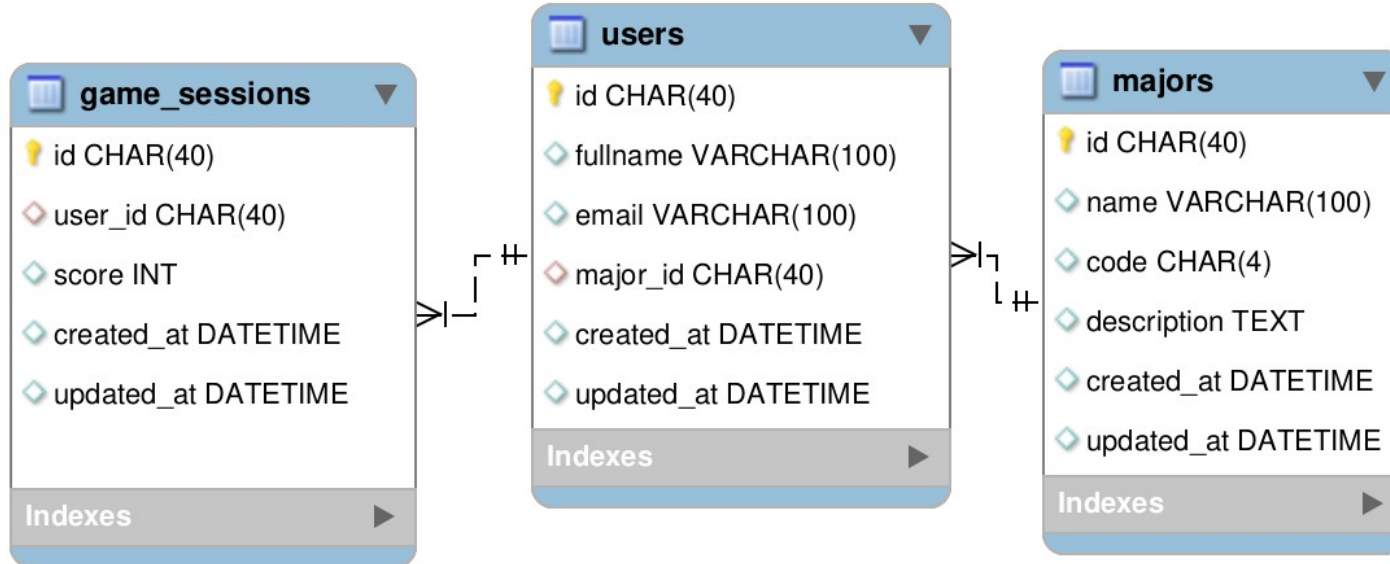




Prof. David Fernandes de Oliveira
Instituto de Computação
UFAM

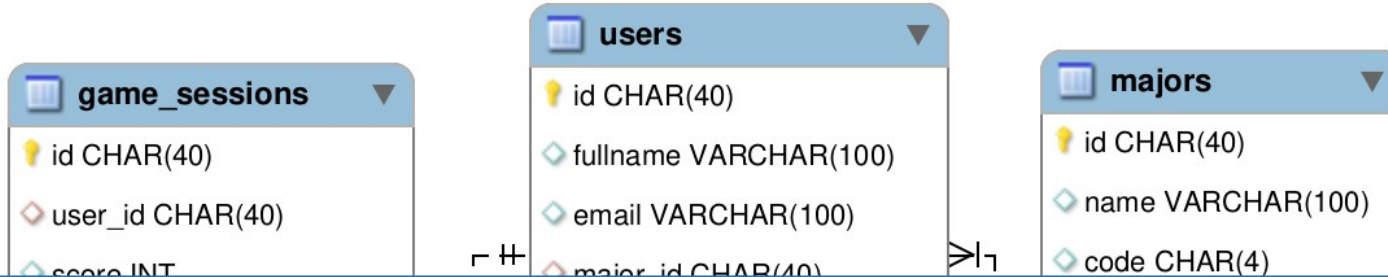
Modelando a Aplicação

- A aplicação irá conter 3 tabelas: **users**, **game_sessions** (partida do jogo), **majors** (curso universitário)



Modelando a Aplicação

- A aplicação irá conter 3 tabelas: **users**, **game_sessions** (partida do jogo), **majors** (curso universitário)



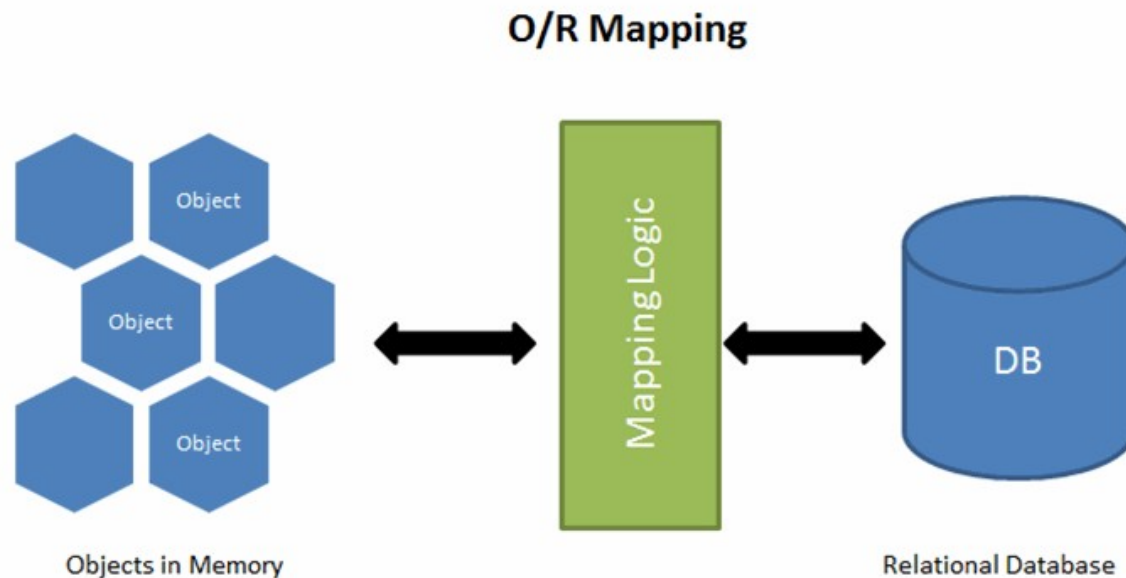
Descrição das Tabelas:

- **Tabela users:** dados dos usuários da aplicação
- **Tabela game_sessions:** dados de cada partida do jogo
- **Tabela majors:** cursos (cc, es, ec, etc) dos jogadores

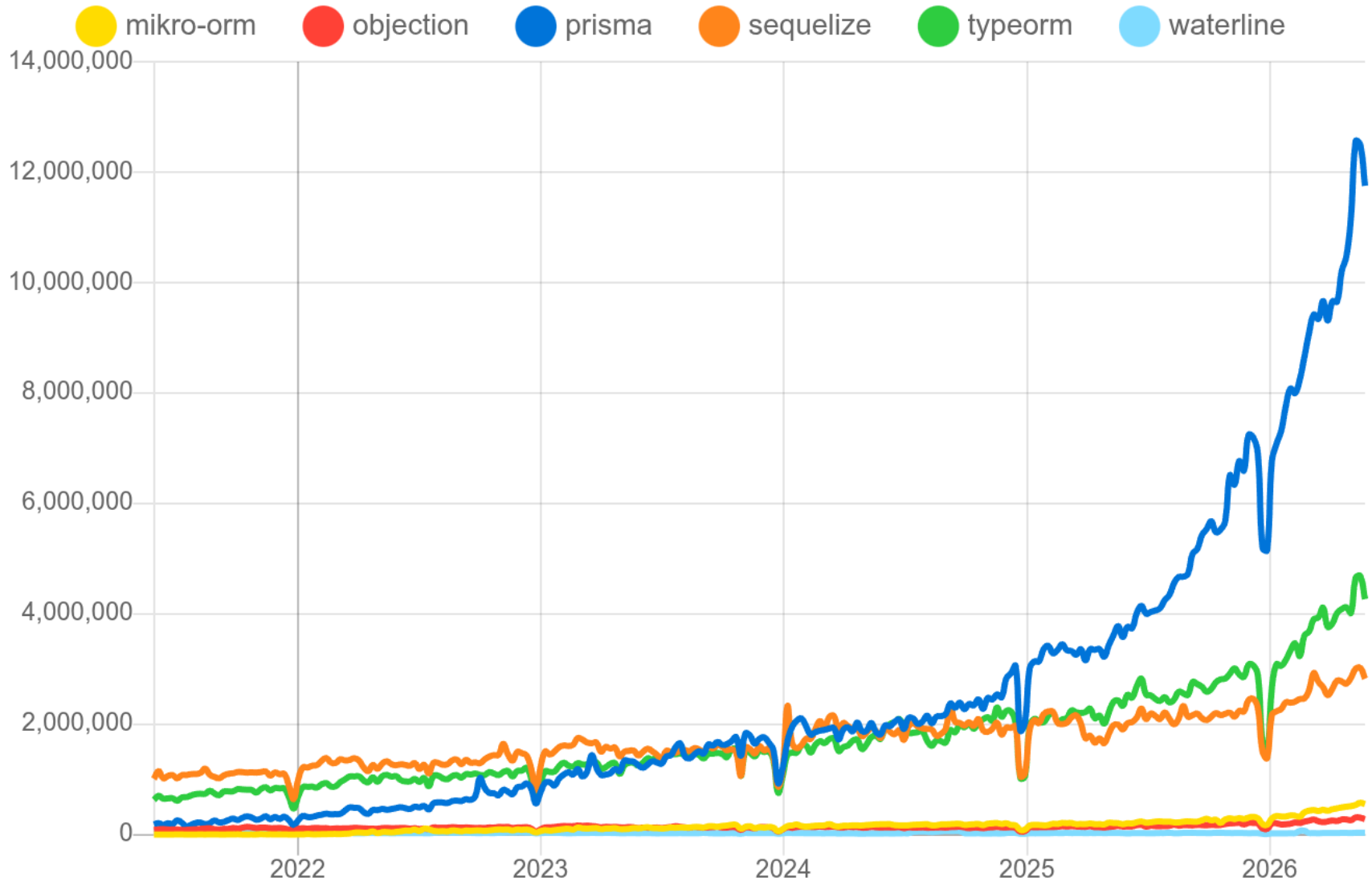


Object Relational Mapper - ORM

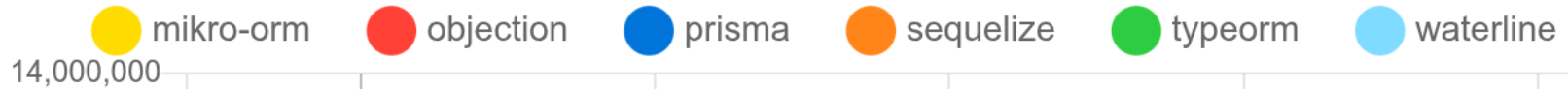
- ORM é uma técnica que permite **consultar e manipular dados** de um database usando o paradigma de orientação a objetos
- Desta forma, o acesso aos dados não é feito através da linguagem SQL, e sim através de objetos



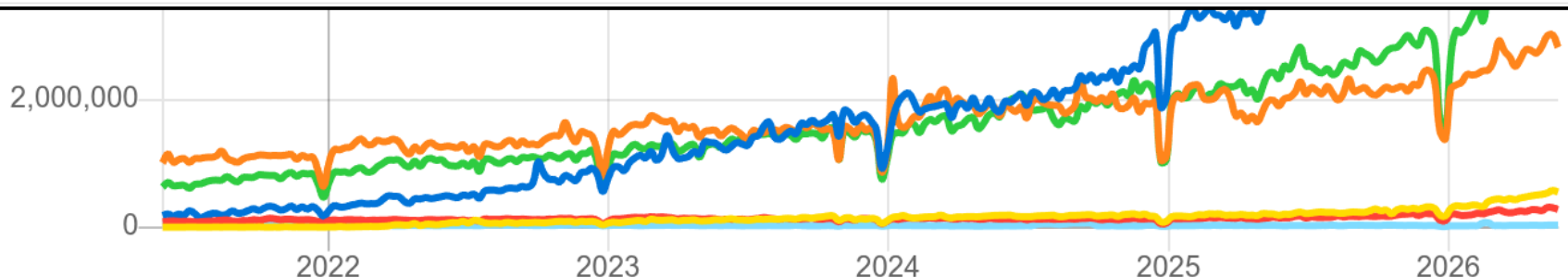
Escolhendo o ORM



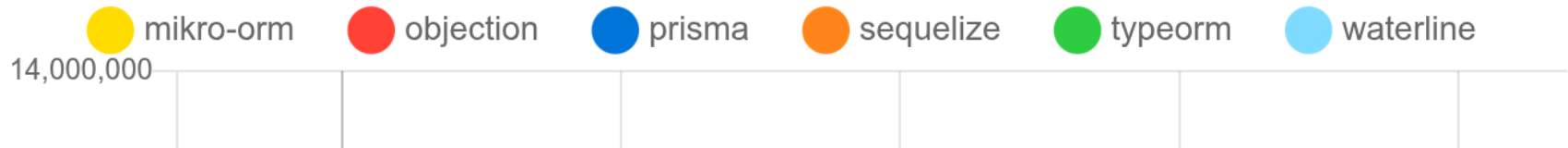
Escolhendo o ORM



			Stars	Issues	Version	Updated ?	Created ?	Size
	mikro-orm	 	-	-	7.1.3	2 days ago	8 years ago	install size 
	objection	 	7,341	129	3.1.5	2 years ago	14 years ago	install size 
	prisma	 	46,043	2,625	7.8.0	a month ago	10 years ago	install size 
	sequelize	 	30,349	1,033	6.37.8	3 months ago	15 years ago	install size 
	typeorm	 	36,522	546	1.0.0	14 days ago	10 years ago	install size 
	waterline	 	5,392	33	0.15.2	3 years ago	13 years ago	install size 

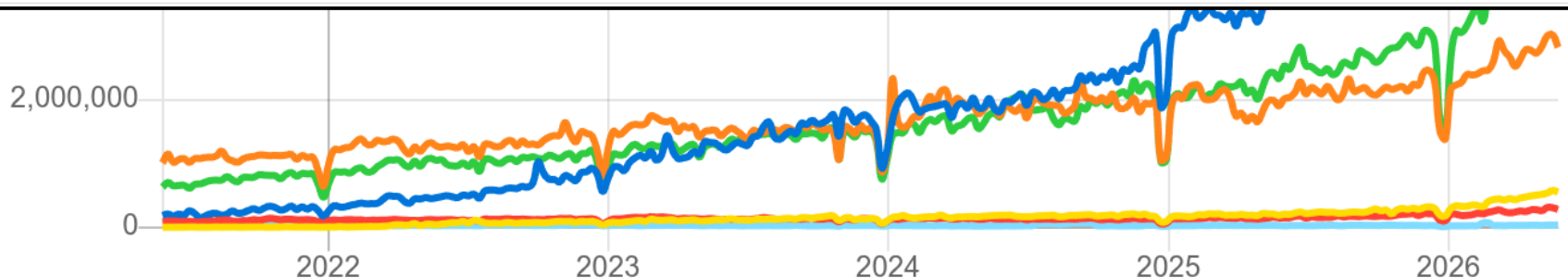


Escolhendo o ORM



	Stars	Issues	Version	Updated ?	Created ?	Size
 mikro-orm  	-	-	7.1.3	2 days ago	8 years ago	install size 12.4 kB
 objection  	7,341	129	3.1.5	2 years ago	14 years ago	install size 1.84 MB
 prisma  	46,043	2,625	7.8.0	a month ago	10 years ago	install size 172 MB
 sequelize  	30,230	1,058	6.21.0	15 years ago	15 years ago	install size 1.84 MB
 typeorm  	36,522	546	1.0.0	14 days ago	10 years ago	install size 22.4 MB
 waterline  	5,392	33	0.15.2	3 years ago	13 years ago	install size 6.35 MB

Sistema de ORM escolhido:  **Prisma**





 PRISMA ORM

ORM de última geração para Node.js e TypeScript

O Prisma ORM aprimora a experiência do desenvolvedor com modelagem de dados intuitiva, migrações automatizadas e segurança de tipos.

Leia a documentação →



45k+

Estrelas no GitHub



250k+

Desenvolvedores ativos



- Para seguirmos, vamos criar um container Docker com o SGBD MySQL e outro com o PhpMyAdmin para acesso ao banco
- Para que o PhpMyAdmin acesse ao banco, precisamos criar uma rede Docker para que os dois containers se comuniquem

```
$ docker network create game-app-network
```

A screenshot of a terminal window with a dark background. The window title is "~/d/p/ExpTs". The prompt is "david@coyote ~/dev/pw/ExpTs [main]". The command entered is "\$ docker network create game-app-network". The output is a long alphanumeric string: "d66de3cca00d8ba15cc09b1bec3c083dd30367296ae63e0739df12e9ce403b69". The prompt is then "\$".

```
~/d/p/ExpTs
david@coyote ~/dev/pw/ExpTs [main] $ docker network create game-app-network
d66de3cca00d8ba15cc09b1bec3c083dd30367296ae63e0739df12e9ce403b69
david@coyote ~/dev/pw/ExpTs [main] $
```



- Agora vamos criar um container Docker com o SGBD MySQL

```
$ docker run -d \  
  --name game-app-db \  
  --network game-app-network \  
  -p 3307:3306 \  
  -e MYSQL_ROOT_PASSWORD=senhasegura \  
  -e MYSQL_DATABASE=game \  
  -v game-app-volume:/var/lib/mysql \  
  mysql:latest
```

- Os dados de acesso ao banco de dados criado são:
 - **Porta:** 3307
 - **Nome do banco:** game
 - **Host:** 127.0.0.1
 - **Senha do root:** senhasegura



- Para acessar o banco de dados criados, vamos criar um container contendo uma instância do PhpMyAdmin

```
$ docker run -d \  
  --name game-app-phpmyadmin \  
  --network game-app-network \  
  -e PMA_HOST=game-app-db \  
  -e PMA_PORT=3306 \  
  -e PMA_USER=root \  
  -e PMA_PASSWORD=senhasegura \  
  -p 8081:80 \  
  phpmyadmin/phpmyadmin
```





localhost:8081 / mysql-g... x +

localhost:8081/index.php?route=/server/databases

phpMyAdmin

Recente Favoritos

Novo
game
information_schema
mysql
performance_schema
sys

Servidor: mysql-game-app:3306

Base de Dados SQL Estado Contas de utilizador Exportar Mais

Base de Dados

Criar base de dados

Nome da base de dados utf8mb4_0900_ai_ci **Criar**

☐ Marcar todos **Eliminar**

Base de Dados	Agrupamento (Collation)	Primary replication	Ação
☐ game	utf8mb4_0900_ai_ci	✓ Replicado	Verificar Privilégios
☐ information_schema	utf8mb3_general_ci	✓ Replicado	Verificar Privilégios
☐ mysql	utf8mb4_0900_ai_ci	✓ Replicado	Verificar Privilégios
☐ performance_schema	utf8mb4_0900_ai_ci	✓ Replicado	Verificar Privilégios
☐ sys	utf8mb4_0900_ai_ci	✓ Replicado	Verificar Privilégios

Total: 5

localhost:8081/index.php?route=/server/privileges&viewing_mode=server

phpMyAdmin

express JS



- Use os comandos abaixo para instalar o **Prisma** como dependência de desenvolvimento

```
$ npm install -D prisma  
$ npm install @prisma/client @prisma/adapter-mariadb
```

- Após isso, execute o comando abaixo para que o Prisma crie os arquivos iniciais de configuração

```
$ npx prisma init --datasource-provider mysql
```



```
david@coyote:~/dev/pw/ExpTS
david@coyote ~/dev/pw/ExpTS $ npx prisma init --datasource-provider mysql
Initialized Prisma in your project

prisma/
├── schema.prisma
└── prisma.config.ts

$ npm install -D prisma
$ npm install @prisma/client @prisma/adapter-mariadb

warn You already have a .gitignore file. Don't forget to add .env in it to
not commit any private information.

Next, set up your database:
1. Configure your DATABASE_URL in prisma.config.ts
2. Run prisma db pull to introspect your database.

Then, define your models in prisma/schema.prisma and run prisma migrate de
v to apply your schema.

Learn more: https://pris.ly/getting-started

david@coyote ~/dev/pw/ExpTS $
```

Node 24.15.0



- O comando **prisma init** adiciona uma variável `DATABASE_URL` no arquivo **.env**, contendo uma string de conexão com o banco
- Será preciso editar o valor dessa variável conforme a realidade do banco de dados utilizado

```
DATABASE_URL="mysql://root:senhasegura@127.0.0.1:3307/game"
```

SGBD

usuário
do
banco

senha
do
usuário

host

porta

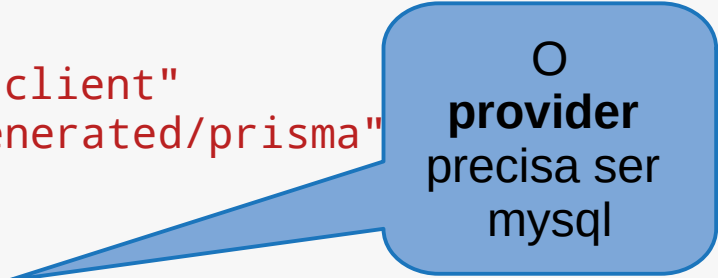
nome
do
banco



- O comando **prisma init** também cria um arquivo **prisma/schema.prisma**, onde serão criados os modelos da aplicação
- Nesse arquivo, é importante que **provider** seja **mysql**, conforme mostrado abaixo

```
generator client {  
  provider = "prisma-client"  
  output = "../src/generated/prisma"  
}
```

```
datasource db {  
  provider = "mysql"  
}
```

A blue rounded rectangular callout box with a pointer directed at the "mysql" value in the code block above. It contains the text "O provider precisa ser mysql".

O
provider
precisa ser
mysql



The screenshot shows the Visual Studio Code interface with the Extensions Marketplace open. The search bar contains 'prisma'. The first result, 'Prisma' by prisma.io, is highlighted. A blue speech bubble points to the extension's icon, containing the text: 'Para usar o Prisma, é altamente recomendado instalar a extensão **Prisma**, do **Visual Studio**'.

Prisma
Prisma [prisma.io](#) | 3,336,261 | ★★★★★ (30)
Adds syntax highlighting, formatting, auto-completion, jump-to-definition and linting for prisma files.

Uninstall Auto Update

Prisma VS
Adds a database highlighting, linting, formatting, jump-to-definition and more.

Features

Extension	Downloads	Rating	Author	Install
Prisma	9K	★ 5	Quernest	Install
Prisma - Insider Build	251K	★ 5	Prisma	Install
Prisma Next	133K	★ 5	William Luke	Install
Prisma Generator	38K	★ 5	AbianS	Install
Prisma Import	16K	★ 5		Install

Adicionando um modelo

- Ainda no arquivo **prisma/schema.prisma**, vamos adicionar o primeiro modelo de nossa aplicação

```
model Major {  
  id          String    @id @default(uuid()) @db.Char(36)  
  name        String    @unique @db.VarChar(100)  
  code        String    @unique @db.Char(4)  
  description  String?   @db.Text()  
  createdAt   DateTime   @default(now()) @map("created_at")  
  updatedAt   DateTime   @updatedAt @map("updated_at")  
  
  @@map("majors")  
}
```

Adicionando um modelo

- Após isso, usamos o comando **npx prisma migrate dev --name** para gerar a **migração** e criar a tabela **majors**

```
david@coyote:~/dev/pw/ExpTS
david@coyote ~/dev/pw/ExpTS $ npx prisma migrate dev --name create-major-table

Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.
Datasource "db": MySQL database "game" at "localhost:3407"

Applying migration `20260602175411_create_major_table`

The following migration(s) have been applied:

migrations/
├─ 20260602175411_create_major_table/
│   └─ migration.sql

Your database is now in sync with your schema.

david@coyote ~/dev/pw/ExpTS $
```

Adicionando um modelo

- Após isso, usamos o comando **npx prisma migrate dev --name** para gerar a **migração** e criar a tabela **majors**

```
david@coyote:~/dev/pw/ExpTS
david@coyote ~/dev/pw/ExpTS $ npx prisma migrate dev --name create-major-table
Loaded Prisma config from prisma.config.ts.
```

Tabela	Ação	Registos	Tipo
☐ majors	★ Procurar Estrutura Pesquisar Inserir Limpa Eliminar	0	InnoDB
☐ _prisma_migrations	★ Procurar Estrutura Pesquisar Inserir Limpa Eliminar	1	InnoDB
2 tabelas	Soma	1	InnoDB

```
The following migration(s) have been applied:

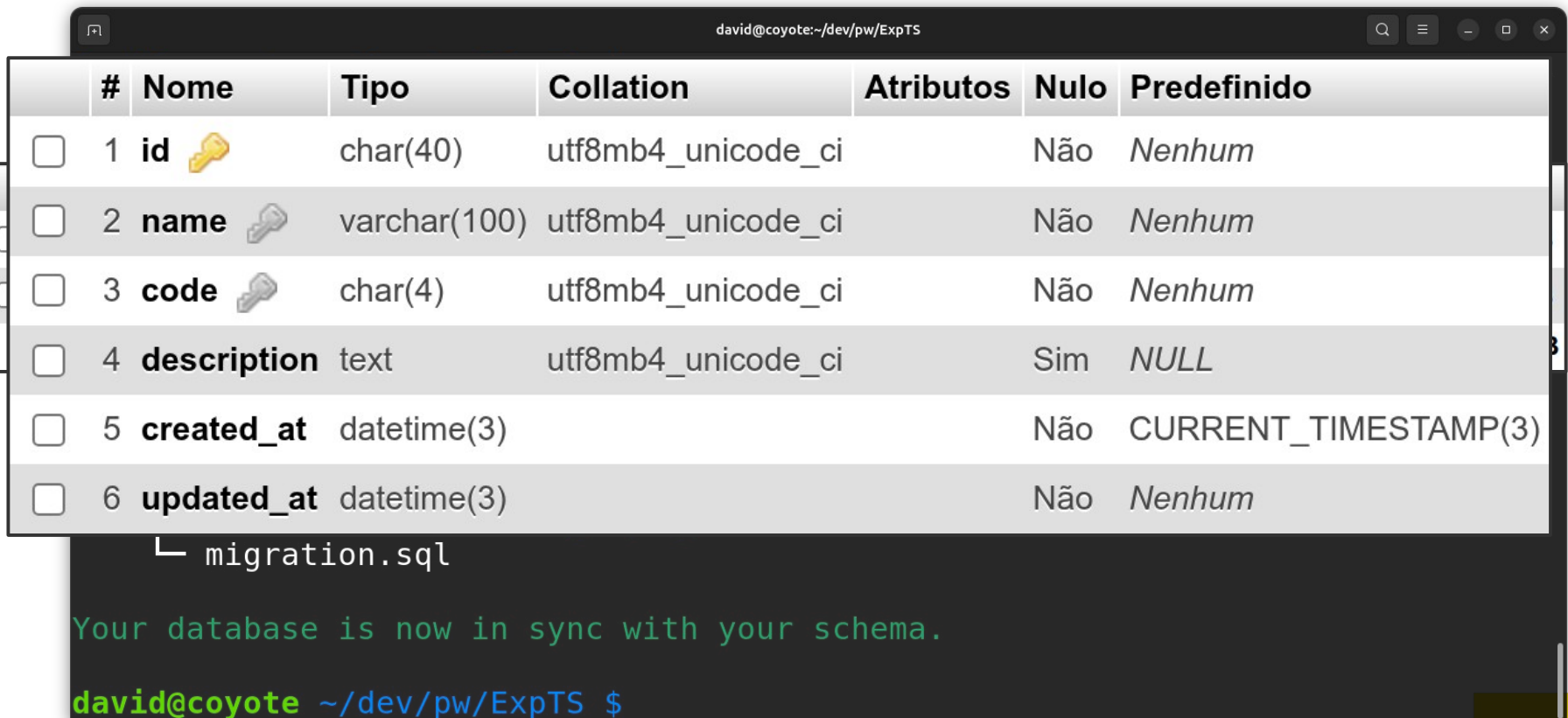
migrations/
├─ 20260602175411_create_major_table/
│  └─ migration.sql

Your database is now in sync with your schema.

david@coyote ~/dev/pw/ExpTS $
```

Adicionando um modelo

- Após isso, usamos o comando **npx prisma migrate dev --name** para gerar a **migração** e criar a tabela **majors**



```
david@coyote:~/dev/pw/ExpTS
```

#	Nome	Tipo	Collation	Atributos	Nulo	Predefinido
☐ 1	id 🔑	char(40)	utf8mb4_unicode_ci		Não	Nenhum
☐ 2	name 🔑	varchar(100)	utf8mb4_unicode_ci		Não	Nenhum
☐ 3	code 🔑	char(4)	utf8mb4_unicode_ci		Não	Nenhum
☐ 4	description	text	utf8mb4_unicode_ci		Sim	NULL
☐ 5	created_at	datetime(3)			Não	CURRENT_TIMESTAMP(3)
☐ 6	updated_at	datetime(3)			Não	Nenhum

```
└─ migration.sql
```

Your database is now in sync with your schema.

```
david@coyote ~/dev/pw/ExpTS $
```

Adicionando um modelo

- Depois é preciso rodar **npx prisma generate**, para o prisma cliente criar as classes e tipos usados para acessar o banco

```
david@coyote:~/dev/pw
david@coyote ~/dev/pw on main $ npx prisma generate
Loaded Prisma config from prisma.config.ts.

Prisma schema loaded from prisma/schema.prisma.

✓ Generated Prisma Client (7.8.0) to ./src/generated/prisma in 28ms
david@coyote ~/dev/pw on main $
```

Exercício

Tabela	Ação	Registos	Tipo
☐ majors	★ Procurar Estrutura Pesquisar Inserir Limpa Eliminar	0	InnoDB
☐ _prisma_migrations	★ Procurar Estrutura Pesquisar Inserir Limpa Eliminar	1	InnoDB

#	Nome	Tipo	Collation	Atributos	Nulo	Predefinido
☐ 1	id 🔑	char(40)	utf8mb4_unicode_ci		Não	Nenhum
☐ 2	name 🔑	varchar(100)	utf8mb4_unicode_ci		Não	Nenhum
☐ 3	code 🔑	char(4)	utf8mb4_unicode_ci		Não	Nenhum
☐ 4	description	text	utf8mb4_unicode_ci		Sim	NULL
☐ 5	created_at	datetime(3)				NOT CURRENT_TIMESTAMP(3)
☐ 6	updated_at	datetime(3)				

Instale e configure o **Prisma** em sua aplicação. Após isso, crie o modelo **Major** e rode a migration para que a tabela majors seja criada no banco de dados.



Encontrando registros

- Para recuperar registros no banco de dados:

```
import { PrismaClient, User } from "../generated/prisma/client";
import { PrismaMariaDb } from "@prisma/adapter-mariadb";
const adapter = new PrismaMariaDb(process.env.DATABASE_URL!);
const prisma = new PrismaClient({ adapter });
```

- Exemplos:

```
SELECT * FROM user WHERE nome = 'Carlos':
```

```
await prisma.user.findMany({ where: { nome: 'Carlos' } });
```

```
SELECT nome, idade FROM user WHERE nome = 'Carlos':
```

```
await prisma.user.findMany({
  where: { nome: 'Carlos' },
  select: { nome: true, idade: true }
});
```


Encontrando registros

- Exemplos de consultas com o objeto **Sequelize.Op**

```
SELECT * FROM user WHERE idade < 30:
```

```
await prisma.user.findMany({ where: { idade: { lt: 30 } } });
```

```
SELECT * FROM user WHERE idade >= 21:
```

```
await prisma.user.findMany({ where: { idade: { lte: 21 } } });
```

```
SELECT * FROM user WHERE nome IN ('Carlos', 'Maria'):
```

```
await prisma.user.findMany({  
  where: { nome: { in: ['Carlos', 'Maria'] } }  
});
```

```
SELECT * FROM user WHERE nome NOT IN ('Carlos', 'Maria'):
```

```
await prisma.user.findMany({  
  where: { nome: { notIn: ['Carlos', 'Maria'] } }  
});
```

Encontrando registros

- Seleccionando usuários cujo nome possuem a string Carlos

```
SELECT * FROM user WHERE nome like '%Carlos%':
```

```
await prisma.user.findMany({  
  where: { nome: { contains: 'Carlos' } }  
});
```

- Seleccionando usuários cujo nome começam com Carlos

```
SELECT * FROM user WHERE nome like 'Carlos%':
```

```
await prisma.user.findMany({  
  where: { nome: { startsWith: 'Carlos' } }  
});
```

- Seleccionando usuários cujo nome terminam com Oliveira

```
SELECT * FROM user WHERE nome like '%Oliveira':
```

```
await prisma.user.findMany({  
  where: { nome: { endsWith: 'Oliveira' } }  
});
```

Paginação

- As cláusulas **skip** e **take** podem ser usadas em conjunto para contruir um sistema de paginação

- Seleccionando os usuários 1-10 com idade maior que 20

```
SELECT * FROM user WHERE idade > 20 LIMIT 10 OFFSET 0;
```

```
await prisma.user.findMany({  
  where: { idade: { gt: 20 }},  
  skip: 0, take: 10  
});
```

- Seleccionando os usuários 11-20 com idade maior que 20

```
SELECT * FROM user WHERE idade > 20 LIMIT 10 OFFSET 10;
```

```
await prisma.user.findMany({  
  where: { idade: { gt: 20 }},  
  skip: 10 , take: 10  
});
```

Ordenando Registros

- Os resultados recuperados podem ser ordenados de forma crescente (ASC) ou decrescente (DESC) por qualquer atributo
- Usuários >=18 ordenados pelo nome em ordem crescente

```
SELECT * FROM user WHERE idade >= 18 ORDER BY nome ASC;
```

```
await prisma.user.findMany({  
  where: { idade: { gte: 18 }}, orderBy: { nome, 'asc' }  
});
```

- Usuários >=18 ordenados pelo nome em ordem decrescente

```
SELECT * FROM user WHERE idade >= 18 ORDER BY nome DESC;
```

```
await prisma.user.findMany({  
  where: { idade: { gte: 18 }}, orderBy: { nome, 'desc' }  
});
```

Selecionando apenas um registro

- O comando **findMany()** retorna um array com todos os registros que atenderem os critérios da cláusula **where**
 - Note que, se apenas um registro atender aos critérios da cláusula **where**, será retornado um array com uma única posição
- Uma alternativa é o uso de **findFirst()**, que ao invés de recuperar um array, retorna apenas um objeto

```
SELECT * FROM user WHERE idade >= 18;
```

```
await prisma.user.findFirst({ where: { idade: { lte: 18 } } })
```

Criando novos registros

- Criação de novos registros no banco de dados:

```
await prisma.user.create(data: { initialValues } );
```

- Exemplo:

```
INSERT INTO user (nome, idade) VALUES ('Carlos', 26);
```

```
await prisma.user.create({ data: {nome: 'Carlos', idade: 16}})
```

Atualizando registros existentes

- Atualização de registros já existentes no banco de dados:

```
await prisma.user.update({values},{criteria});
```

- Exemplo:

```
UPDATE user SET idade=30 WHERE nome = 'Carlos';
```

```
await prisma.user.update(  
  { idade: 30 },  
  { where: { nome: 'Carlos' } }  
);
```

Apagando registros existentes

- Apagando registros no banco de dados:

```
await prisma.user.deleteMany(criteria);
```

- Exemplos:

```
DELETE FROM user WHERE nome = 'Carlos';
```

```
await prisma.user.deleteMany({  
  where: { nome: 'Carlos' }  
});
```

```
DELETE FROM user WHERE id IN (3, 97);
```

```
await prisma.user.deleteMany({  
  where: { id: { in: [3, 97] } }  
});
```


Desenvolvendo CRUDs

- Uma parte importante do desenvolvimento de uma aplicação é a criação do **CRUD** de alguns modelos
 - O CRUD de um modelo é um conjunto de páginas responsáveis por quatro operações sobre esse modelo:



Desenvolvendo CRUDs

- Para exemplificar a criação de novos CRUDs, vamos desenvolver um para o modelo **Major**
- O primeiro passo é criar um controlador vazio para o CRUD de **Major** dentro do diretório **/src/controllers**

```
// Arquivo src/controllers/major.ts
```

```
import { Request, Response } from 'express'
```

```
const index = (req: Request, res: Response) => {}
```

```
const create = (req: Request, res: Response) => {}
```

```
const read = (req: Request, res: Response) => {}
```

```
const update = (req: Request, res: Response) => {}
```

```
const remove = (req: Request, res: Response) => {}
```

```
export default { index, read, create, update, remove }
```

Operações
do CRUD

Definindo as Rotas do CRUD

- O segundo passo é definir as rotas para cada operação do CRUD:

```
// Arquivo src/router/router.ts

import express from "express"
import majorController from "../controllers/major"

const router = express.Router();
...
// rotas de MajorController
router.get('/major', majorController.index)
router.all('/major/create', majorController.create)
router.get('/major/read/:id', majorController.read)
router.all('/major/update/:id', majorController.update)
router.post('/major/remove/:id', majorController.remove)
...
export default router
```

Definindo as Rotas do CRUD

- O segundo passo é definir as rotas para cada operação do CRUD:

```
// Arquivo src/router/router.ts
import express from "express"
import majorController from "../controllers/major"

const router = express.Router();
...
// ...
router.get('/major/read/:id', majorController.read)
router.all('/major/update/:id', majorController.update)
router.post('/major/remove/:id', majorController.remove)
...
export default router
```

Embora não faça parte do CRUD, o objetivo da rota **/major** é listar os cursos existentes

Definindo as Rotas do CRUD

- O segundo passo é definir as rotas para cada operação do CRUD:

```
// Arquivo src/router/router.ts
```

```
import express from "express"
```

```
import majorController from "../controllers/major"
```

```
const router = express.Router();
```

```
...
```

```
// ...
```

```
router.get('/major/:id', majorController.read)
```

Note que as rotas para **read**, **update** e **remove** terminam com a string **:id**, que representa um parâmetro utilizado para informar que curso se deseja **ler**, **atualizar** ou **apagar**

```
router.post('/major/remove/:id', majorController.remove)
```

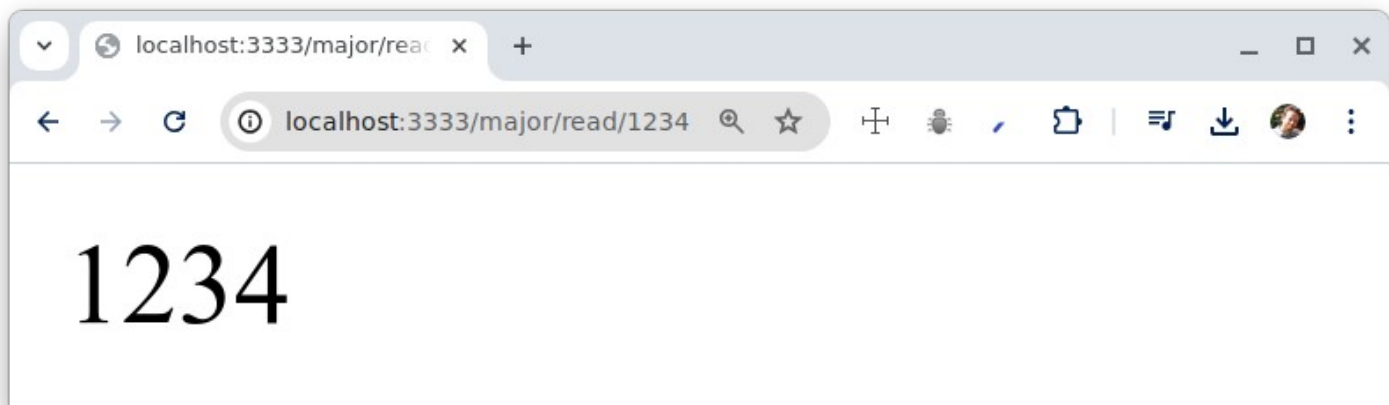
```
...
```

```
export default router
```

Definindo as Rotas do CRUD

- Por exemplo, na url **http://localhost:3333/major/read/1234**, o valor do parâmetro **id** é 1234
- Para ler o valor de **id** dentro da action, podemos usar o atributo **param** de **req** (objeto da requisição do usuário):

```
const read = (req: Request, res: Response) => {  
  const { id } = req.params  
  res.send(id)  
}
```



Requisições POST

- Quando o usuário preenche e submete um formulário POST, os dados informados pelo usuário são enviados para o servidor

```
<form action="/major/create" method="post">  
  <input type="text" name="code">  
  <input type="text" name="name">  
  <input type="submit" value="Enviar">  
</form>
```

- Após a submissão, os dados são enviados através do corpo da requisição HTTP (request body)
- Esses dados são enviados no formato **application/x-www-form-urlencoded**, que também é usado em requisições GET:

```
code=IE08&name=Ciencia+da+Computacao
```

Requisições POST

- O Express possui um middleware nativo chamado **urlencoded**, que pode ser usado para extrair os dados de **request body**
- Para usá-lo, basta inserir a linha abaixo no arquivo **app.js**, em alguma lugar antes da chamada ao middleware router:

```
// Arquivo index.ts  
app.use(express.urlencoded({extended: false}));  
...  
app.use(router);
```


Requisições POST

- Após isso, o **urlencoded** irá extrair os dados do **request body** de todas as requisições POST e copiá-los no objeto **req.body**

```
// Arquivo src/controllers/major.ts
import { createMajor } from '../major.service'

const create = async (req: Request, res: Response) => {
  if (req.method === 'GET') {
    res.render('major/create')
  } else {
    try {
      await createMajor(req.body)
      res.redirect('/major')
    } catch (err) {
      res.status(500).send(err)
    }
  }
}
```

Uso da função `createMajor`, que será definida na camada de serviço

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/services/major.ts
```

```
import { type Major, PrismaClient } from "../../generated/prisma/client";
import { PrismaMariaDb } from "@prisma/adapters-mariadb";
import type { CreateMajorDto } from "../../types/major";

const adapter = new PrismaMariaDb(process.env.DATABASE_URL!);
const prisma = new PrismaClient({ adapter });

export const getAllMajors = async(): Promise<Major[]> => {
  return prisma.major.findMany();
}

export const createMajor = async (
  newMajor: CreateMajorDto
): Promise<Major> => {
  return await prisma.major.create({ data: newMajor })
}
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/services/major.ts
```

```
import { type Major, PrismaClient } from "../../generated/prisma/client";  
import { PrismaMariaDb } from "@prisma/adapters-mariadb";
```

```
import
```

```
const majorAlreadyExists = async (name: string): Promise<boolean>
```

```
const
```

```
const getMajor = async (id: string): Promise<Major>
```

```
export const updateMajor = async (id: string, major: MajorUpdateDto):  
    Promise<[affectedCount: number]>
```

```
}
```

```
export const removeMajor = async (id: string): Promise<number>
```

```
export
```

```
    newMajor: CreateMajorDto
```

```
): Promise<Major> => {
```

```
    return await prisma.major.create({ data: newMajor })
```

```
}
```

Além das funções mostradas, o serviço major precisa ter funções como:

```
const majorAlreadyExists = async (name: string): Promise<boolean>
```

```
const getMajor = async (id: string): Promise<Major>
```

```
const updateMajor = async (id: string, major: MajorUpdateDto):  
    Promise<[affectedCount: number]>
```

```
const removeMajor = async (id: string): Promise<number>
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/services/major.ts
```

```
import { type Major, PrismaClient } from "../../generated/prisma/client";  
import { PrismaMariaDb } from "@prisma/adapters-mariadb";
```

```
import
```

```
const
```

```
const
```

```
export
```

```
return
```

```
}
```

```
export
```

```
newMajor: CreateMajorDto
```

```
): Promise<Major> => {
```

```
  return await prisma.major.create({ data: newMajor })
```

```
}
```

Além das funções mostradas, o serviço major precisa ter funções como:

Uma vantagem do uso de serviços é que suas funções podem ser utilizadas em outras partes da aplicação, diminuindo a réplica de códigos em vários arquivos.

```
Promise<[affectedCount: number]>
```

```
const removeMajor = async (id: string): Promise<number>
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/services/major.ts
```

```
import { type Major, PrismaClient } from "../../generated/prisma/client";  
import { PrismaMariaDb } from "@prisma/adapters-mariadb";
```

```
import
```

```
const createMajor = async (id: string): Promise<Major> => {  
  const major = await prisma.major.create({ data: newMajor });  
  return major;  
}
```

Além das funções mostradas, o serviço major precisa ter funções como:

Uma vantagem do uso de serviços é que suas funções podem ser reutilizadas em outros serviços.

Outra vantagem dos serviços é que, caso se queira mudar o ORM da aplicação, o esforço será muito menor. Isso porque eles serão os únicos arquivos que usam os recursos do ORM para recuperar, atualizar e criar dados.

```
const removeMajor = async (id: string): Promise<number> => {  
  const major = await prisma.major.delete({ where: { id } });  
  return major;  
}
```

```
const createMajor = async (id: string): Promise<Major> => {  
  const major = await prisma.major.create({ data: newMajor });  
  return major;  
}
```

Data Transfer Objects (DTO)

- Os arquivos **types/**/*types.ts** possuem as **interfaces** e **types**, em especial os **DTOs**, usados dentro dos services e controllers
- DTO é uma interface ou type usado para representar os objetos de dados que são trocados entre a API e as aplicações client
 - Por exemplo, para criar um novo produto, a aplicação cliente precisa enviar para a API os dados desse novo produto, sendo o formato desses dados é definido através de um DTO

Data Transfer Objects (DTO)

- Os DTOs geralmente contêm um subconjunto dos atributos de um dado modelo, e para gerá-los podemos usar o comando Pick

```
// Arquivo types/major.ts  
  
import type { Major } from "../generated/prisma/browser";  
  
export type CreateMajorDto =  
  Pick<Major, 'name' | 'code' | 'description'>  
export type UpdateMajorDto =  
  Pick<Major, 'name' | 'code' | 'description'>
```

Cria um novo type contendo apenas as propriedades name, code e description do modelo Major

Data Transfer Objects (DTO)

- Os DTOs são usados principalmente na camada de serviço, mas também podem ser utilizados nos controladores

```
// Arquivo src/services/major.ts
```

```
import { type Major, PrismaClient } from "../generated/prisma/client";
import { PrismaMariaDb } from "@prisma/adapters-mariadb";
import type { CreateMajorDto } from "../types/major";

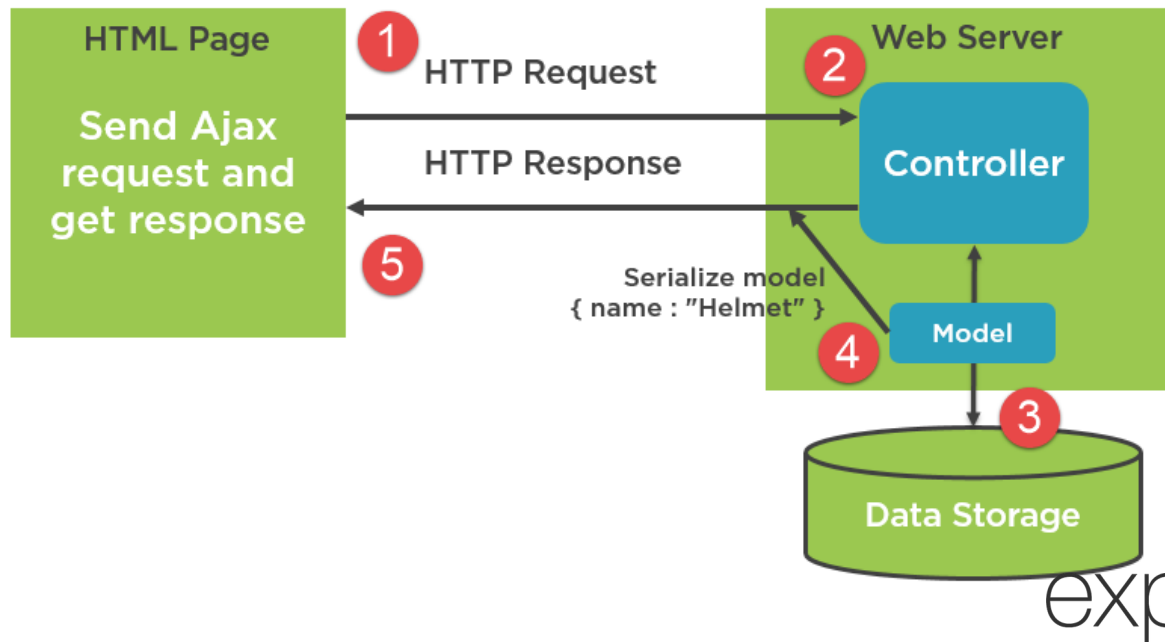
const adapter = new PrismaMariaDb(process.env.DATABASE_URL!);
const prisma = new PrismaClient({ adapter });

export const getAllMajors = async(): Promise<Major[]> => {
  return prisma.major.findMany();
}

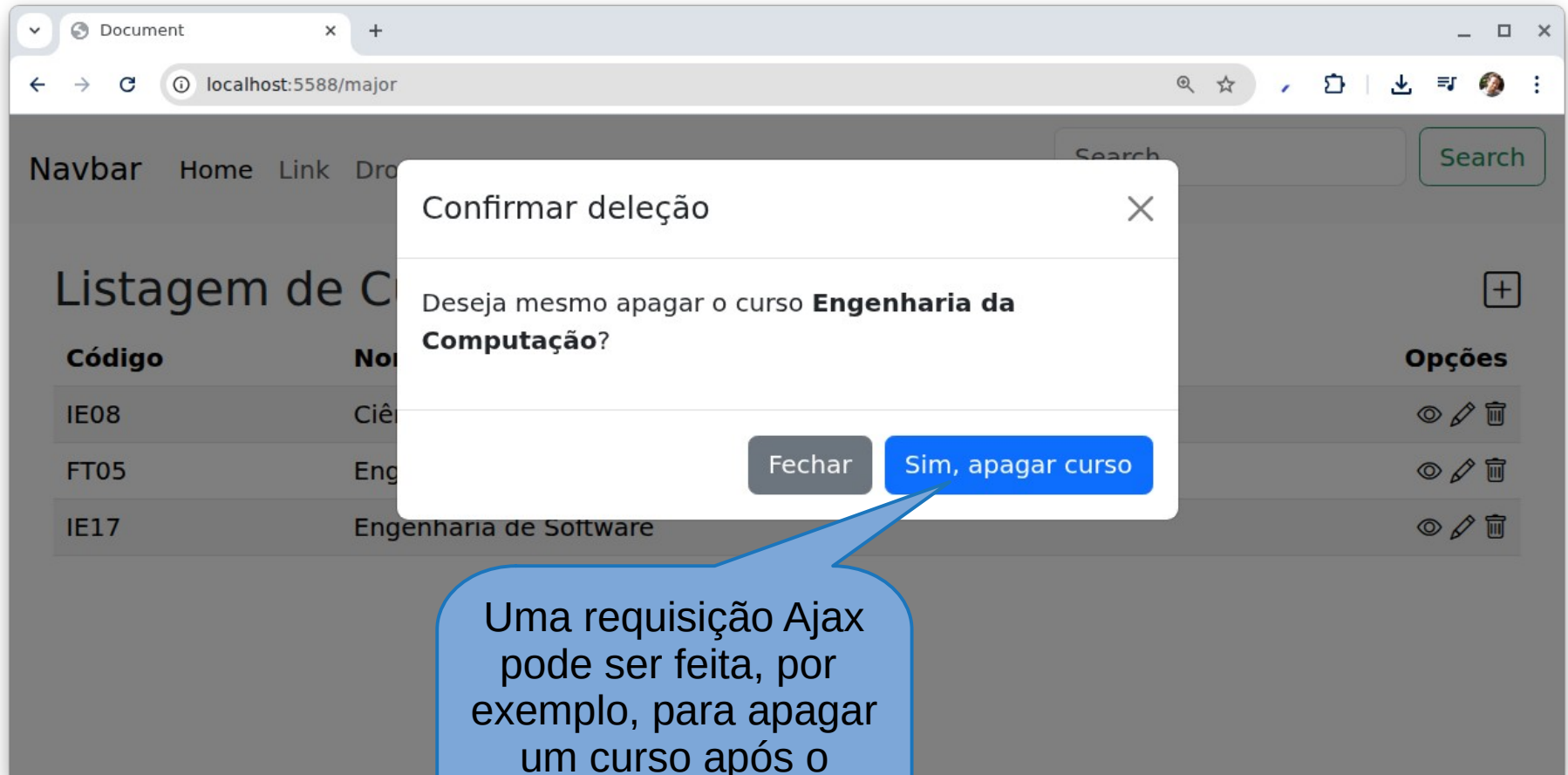
export const createMajor = async (
  newMajor: CreateMajorDto
): Promise<Major> => {
  return await prisma.major.create({ data: newMajor })
}
```


Requisições Ajax

- Ajax significa **Asynchronous JavaScript and XML**
- É uma técnica que permite que páginas web se comuniquem com o servidor em segundo plano, sem recarregar a página
- Permite melhorar a experiência do usuário, tornando aplicações mais rápidas e dinâmicas



Requisições Ajax



Uma requisição Ajax pode ser feita, por exemplo, para apagar um curso após o usuário confirmar a operação clicando em um botão



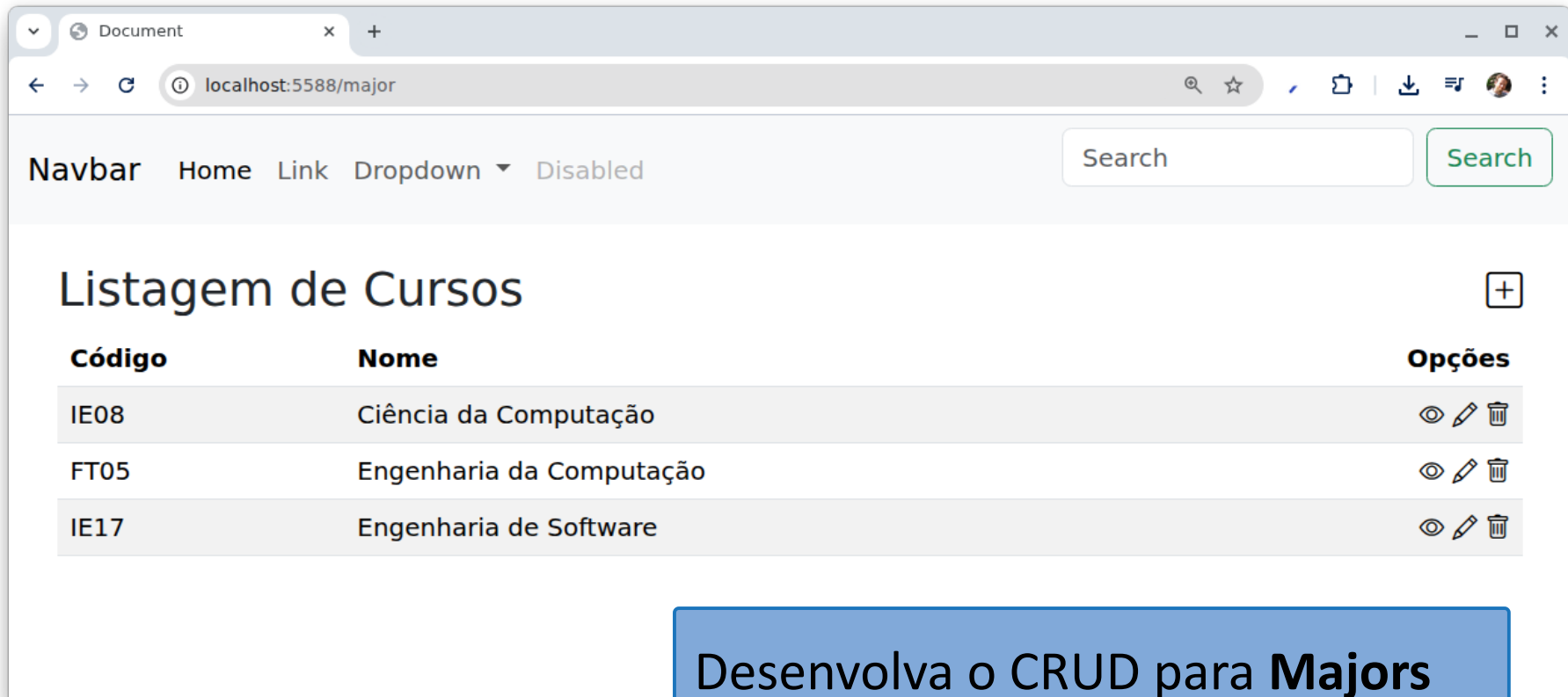
Requisições Ajax

- Usando o código abaixo, o clique aciona a função **deletarMajor(id)**, que faz a requisição Ajax com o servidor

```
<button onclick="deletarMajor('{{id}}')" type="button">  
  Sim, apagar curso  
</button>
```

```
// Arquivo public/js/script.js  
function deletarMajor(majorIdToDelete) {  
  fetch(`/major/remove/${majorIdToDelete}`, { method: 'POST' })  
    .then(response => {  
      if (response.ok) {  
        console.log('Major deletado com sucesso');  
        window.location.reload();  
      } else {  
        console.log('Erro ao deletar');  
      }  
    })  
};
```

Exercício II



Cookies e Sessões

- HTTP é um protocolo que não mantém estado, isto é **não mantém uma conexão**
- Cada requisição que um cliente (browser, insomnia, etc) faz à API é independente das requisições anteriores
- No entanto, muitas aplicações necessitam manter o estado do usuário durante as requests feitas à API
 - Ex: carrinho de compras em sites de comércio eletrônico
- As aplicações possuem duas opções para manter as informações de estado dos clientes:
 - **Cookies**: mantém informações de estado no cliente (browser)
 - **Sessões**: mantém informações de estado no lado servidor

Cookies



- **Cookies** são variáveis enviadas pelo servidor Web para o browser através do **protocolo HTTP**
 - Ficam armazenados no lado cliente
 - São enviados para o servidor em futuros acessos do browser

Requisições do Browser

1 Get index.html HTTP1.1
Host: www.icomp.ufam.edu.br

3 Get index.html HTTP1.1
Host: www.icomp.ufam.edu.br
Cookie: usuario=3452

Requisições de icomp.ufam.edu.br

2 HTTP1.1 200 OK
Set-Cookie: usuario=3452
... Conteúdo da página...

4 HTTP1.1 200 OK
... Conteúdo da página...

Cookies



- Para habilitar o uso de cookies em sua aplicação, é necessário instalar o middleware **cookie-parser**

```
$ npm install cookie-parser  
$ npm install -D @types/cookie-parser
```

- Para usar o middleware, precisamos importar o módulo e adicioná-lo em nossa aplicação com o método **use**

```
// Arquivo src/server.ts  
import cookieParser from 'cookie-parser';  
  
// Método middleware():  
app.use(cookieParser())
```

Cookies



- A partir deste momento, podemos i) criar novos cookies e ii) identificar os cookies enviados pelo browser para o servidor
 - O array **req.cookies** armazena os cookies enviados pelo browser
 - O método **res.cookie** é usado para enviar um pedido de criação de um novo cookie no browser

```
// Arquivo src/router/router.ts  
router.get('/create-cookie', mainController.createCookie);
```

```
// Arquivo src/controllers/main.ts  
const createCookie = function (req: Request, res: Response) {  
  if (!('nomeCookie' in req.cookies)) {  
    res.cookie('nomeCookie', 'valorCookie');  
    res.send('Você NUNCA passou por aqui!');  
  } else {  
    res.send('Você JÁ passou por aqui!');  
  }  
});
```




Você NUNCA
passou por
aqui

Requisição
para criação
do novo
cookie

Ao acessar a página pela
primeira vez, aparece a
mensagem: **Você nunca
passou por aqui.**

localhost:3333/create-coc x +

localhost:3333/create-cookie

Elements Network >>

2

Preserve log Disable cache

No throttling

Filter Invert Hide data URLs

All Fetch/XHR JS CSS Img Media Font Doc WS Wasm Manifest

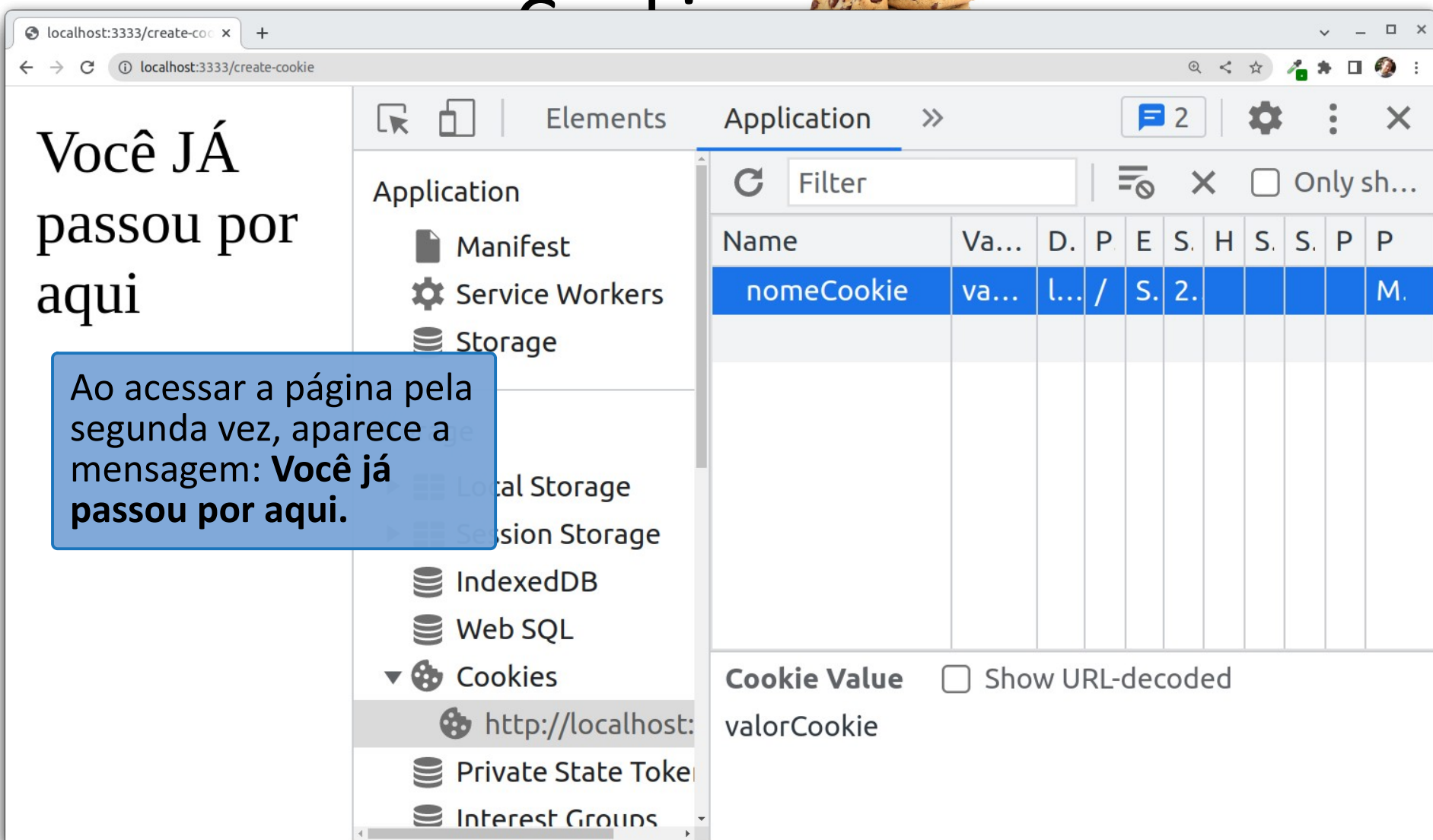
Has blocked cookies

20 ms 40 100 ms

Set-Cookie: nomeCookie=valorCookie; Path=/

1 request:

X-Powered-By: Express



Cookies



- Também podemos criar cookies com data de expiração

```
// Expira 360000 ms (6 minutos) após ser criado  
res.cookie(name, 'value', { maxAge: 360000 } );
```

- Usamos a função **clearCookie** para apagar um cookie já criado

```
// Arquivo src/router/router.ts  
router.get('/clear-cookie', mainController.clearCookie);
```

```
// Arquivo src/controllers/main.ts  
const clearCookie = function (req: Request, res: Response) {  
  res.clearCookie('nomeCookie');  
  res.send('cookie apagado');  
});
```

Cookies



- Também podemos criar cookies com data de expiração

```
// Expira 360000 ms (6 minutos) após ser criado  
res.cookie(name, 'value', { maxAge: 360000 } );
```

- Usamos a função **clearCookie** para apagar um cookie já criado

```
// ...  
route
```

Se o cookie for criado sem data de expiração, ele será apagado após o fechamento da janela do browser

```
// Arquivo src/controllers/main.ts  
const clearCookie = function (req: Request, res: Response) {  
  res.clearCookie('nomeCookie');  
  res.send('cookie apagado');  
});
```

Sessões

- Através de sessões, podemos armazenar informações de estado (variáveis) no lado servidor
- Em vez do browser guardar um cookie por dado, ele guarda apenas um cookie contendo um **id de sessão (connect.sid)**

Requisições do Browser

1 Get index.html HTTP1.1
Host: server.com

3 Get index.html HTTP1.1
Host: server.com
Cookie: connect.sid=6Bh

Requisições de server.com

2 HTTP1.1 200 OK
Set-Cookie: connect.sid=6Bh
... Conteúdo da página...

4 HTTP1.1 200 OK
... Conteúdo da página...

Sessão 6Bh Dados da sessão no servidor

user: Bruna
itens-carrinho: 3
start: 15:30



Sessões

- Para usarmos as sessões, precisamos instalar um módulo para geração de **valores únicos para os IDs** das sessões
- Uma opção é o módulo **uuid** – Universally Unique Identifier – que é uma implementação do UUID descrito na [RFC 4122](https://tools.ietf.org/html/rfc4122)

```
$ npm install uuid
```

- Os UUIDs são valores de 128 bits que podem ser usados como ID únicos de qualquer coisa em sistemas computacionais
 - Ex: f0221c72-ac30-4796-83f5-fd7a8a4f6b15
- Embora a probabilidade de um UUID ser duplicado não seja nula, ela é próximo o suficiente de zero e pode ser ignorada

Sessões

- Para usarmos as sessões, precisamos instalar um módulo para geração de **valores únicos para os IDs** das sessões
- Uma opção é o módulo **uuid** – Universally Unique Identifier – que é uma implementação do UUID descrito na [RFC 4122](https://tools.ietf.org/html/rfc4122)

```
$ npm install uuid
```

- Os UUIDs são valores de 128 bits que podem ser usados como
Alguns desenvolvedores preferem usar o uuid como **chave primária** de tabelas, ao invés de um ID auto incrementado.
- Embora a probabilidade de um UUID ser duplicado não seja nula, ela é próximo o suficiente de zero e pode ser ignorada

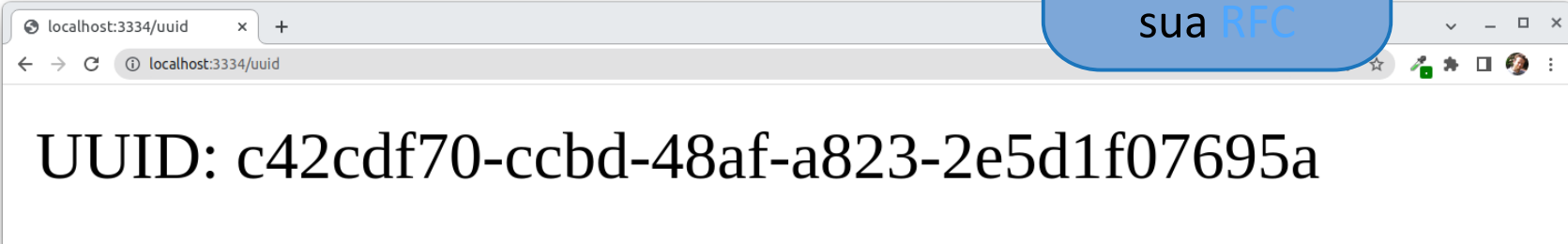
Sessões

- O código abaixo mostra como os UUIDs podem ser gerados a partir do módulo **uuid**

```
// Arquivo src/router/router.ts  
router.get('/uuid', mainController.uuid);
```

```
// Arquivo src/controllers/main.ts  
const uuid = (req, res, next) => {  
  const uniqueId = uuidv4();  
  res.send(`UUID: ${uniqueId}`);  
};
```

Existem 5
versões de
UUID, conforme
descrito em
sua [RFC](#)



UUID: c42cdf70-ccbd-48af-a823-2e5d1f07695a

Sessões

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session  
$ npm install -D @types/express-session
```

- Para usar o middleware, precisamos importá-lo e adicioná-lo em nossa aplicação com o método **use**

```
// Arquivo src/index.ts  
import session from 'express-session';  
...  
app.use(session({  
  genid: () => uuidv4(), // usamos UUID para gerar os SESSID  
  secret: 'Hi9Cf#mK98',  
  resave: true,  
  saveUninitialized: true,  
}));
```

Sessões

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session
$ npm install -D @types/express
```

- Para usar o middleware, precisamos adicioná-lo em nossa aplicação com o seguinte código:

```
// Arquivo src/index.ts
import session from 'express-session'
...
app.use(session({
  genid: () => uuidv4(), // usamos UUID para gerar os SESSID
  secret: 'Hi9Cf#mK98',
  resave: true,
  saveUninitialized: true,
}));
```

Usado para adicionar uma assinatura (similar ao checksum) ao **session.id** enviado para o usuário. Quando o usuário devolve o **session.id**, a assinatura é usada para checar se o **session.id** é válido. Usa uma técnica chamada HMAC.

Sessões

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session
$ npm install -D express-session
```

- Para usar o middleware em nossa aplicação

```
// Arquivo src/index.js
import session from 'express-session'
...
app.use(session({
  genid: () => '...',
  secret: 'Hi9Cf#mK98',
  resave: true,
  saveUninitialized: true,
}));
```

Quando **true**, a sessão do usuário é salva a cada requisição, mesmo que os dados da sessão não tenham sido modificados durante a requisição. Isso mantém a sessão ativa, visto que ela pode ser deletada após algum tempo de desuso.

Quando **session.id** é adicionado ao objeto de configuração, ele adiciona um **session.id** ao usuário. Isso envolve o uso de uma assinatura HMAC para garantir a segurança da sessão.

Para gerar os SESSID

Sessões

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session
$ npm install -D express-session
```

- Para usar o middleware **express-session** em nossa aplicação, precisamos adicioná-lo

```
// Arquivo src/index.js
import session from 'express-session'
...
app.use(session({
  genid: () => {
    secret: 'HjCf#mK98',
    resave: true,
    saveUninitialized: true,
  }
}));
```

Quando **true**, a sessão do usuário é salva a cada requisição.

Quando **true**, força que as sessões não inicializadas sejam salvas no store. Uma sessão não inicializada ocorre quando a sessão é nova e ainda não foi modificada.

Adicionar **session.id** para condicioná-lo

Ário. envolve o que é se o sa uma MAC.

gerar os SESSID

Sessões

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session
```

The screenshot shows a web browser window with the address bar at `localhost:3333/major/create`. The left sidebar contains a form titled "Criação de Curso" with fields for "Name" and "Sigla", and an "Enviar" button. The main content area is the "Application" tab, which displays a table of cookies. The table has columns: Name, Value, Domain, Path, Expires / Max-Age, Secure, HttpOnly, SameSite, and Partition. Two cookies are listed: "SSID" and "connect.sid". The "connect.sid" cookie is highlighted, and its value is shown in the "Cookie Value" section below the table.

Name	Value	Domain	Path	Expires / Max-Age	Secure	HttpOnly	SameSite	Partition
SSID	s...	loc...	/	2024-07-15T...	1.	✓		S...
connect.sid	s...	loc...	/	Session	9.	✓		M...

Cookie Value ☐ Show URL-decoded

s%3A72cd6568-2fed-48f2-a102-a25644fdb63e.EpEr0bvzy%2B0M1CbzwI2Z1B40m11dlJm3qzViZ9tPBZM

```
}});
```

Cadastro de Usuários

- O cadastro de usuário requer a criação do modelo User, que por sua vez se relaciona com o modelo Major

```
model User {  
  id          String      @id @default(uuid()) @db.Char(40)  
  name        String      @db.VarChar(100)  
  email       String      @unique @db.VarChar(100)  
  password    String      @db.Char(60)  
  major       Major       @relation(fields: [majorId], references: [id])  
  majorId     String      @db.Char(40)  
  createdAt   DateTime    @default(now()) @map("created_at")  
  updatedAt   DateTime    @updatedAt @map("updated_at")  
  @@map("usuarios")  
}  
  
model Major {  
  id          String      @id @default(uuid()) @db.Char(40)  
  name        String      @unique @db.VarChar(100)  
  code        String      @unique @db.Char(40)  
  description  String?    @db.Text()  
  createdAt   DateTime    @default(now()) @map("created_at")  
  updatedAt   DateTime    @updatedAt @map("updated_at")  
  User        User[]  
  @@map("majors")  
}
```

Cadastro de Usuários

- O cadastro de usuário requer a criação do modelo User, que por sua vez se relaciona com o modelo Major

```
model User {  
  id          String      @id @default(uuid()) @db.Char(40)  
  name        String      @db.VarChar(100)  
  email       String      @unique @db.VarChar(100)  
  password    String      @db.Char(60)  
  major       Major       @relation(fields: [majorId], references: [id])  
  majorId     String      @db.Char(40)  
  createdAt   DateTime     @default(now())  
  updatedAt   DateTime     @updatedAt  
  @@map("usuarios")  
}  
  
model Major {  
  id          String      @id @default(uuid()) @db.Char(40)  
  name        String      @unique @db.VarChar(100)  
  code        String      @unique @db.Char(40)  
  description  String?     @db.Text()  
  createdAt   DateTime     @default(now()) @map("created_at")  
  updatedAt   DateTime     @updatedAt @map("updated_at")  
  User        User[]  
  @@map("majors")  
}
```

Os campos de relação definem conexões entre modelos no nível Prisma e não existem no banco de dados. Esses campos são usados para gerar o Prisma Client.


```

david@coyote ~/dev/expApi/backend [main]× $ npx prisma migrate dev
--name add-usuarios-tables
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Datasource "db": MySQL database "loja" at "localhost:3306"

model Usuario {
  id          String      @id @default(uuid()) @db.Char(40)
  nome        String      @db.VarChar(100)
  email       String      @unique @db.VarChar(100)
  senha       String      @db.VarChar(100)
  tipoUsuario TipoUsuario @relation(fields=[id], references=[id])
  tipoUsuarioId String    @db.Char(40)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
}

migrations/
└─ 20231028115757 add_usuarios_tables/
   └─ migration.sql

Your database is now in sync with your schema.

model TipoUsuario {
  id          String      @id @default(uuid()) @db.Char(40)
  nome        String      @db.VarChar(100)
  usuarios    Usuario[]
}

@@map("tipos_usuarios")
}

✓ Generated Prisma Client (v5.4.2) to ./node_modules/@prisma/client in 129ms

david@coyote ~/dev/expApi/backend [main]× $

```

- O cadastro de usuário envolve dois modelos Prisma, que
 - Prisma schema loaded from prisma/schema.prisma

Datasource "db": MySQL database "loja" at "localhost:3306"

```
model Usuario {
```

Applying migration `20231028115757 add\_usuarios\_tables`

```
  id          String      @id @default(uuid()) @db.Char(40)
```

```
  nome        String      @db.VarChar(100)
```

```
  email       String      @unique @db.VarChar(100)
```

The following migration(s) have been created and applied from new schema changes:

```
  tipoUsuario TipoUsuario @relation(fields=[id], references=[id])
```

```
  tipoUsuarioId String    @db.Char(40)
```

```
  createdAt   DateTime    @default(now())
```

```
  updatedAt   DateTime    @updatedAt
```

```
└─ 20231028115757 add_usuarios_tables/
```

```
   └─ migration.sql
```

```
}
```

Your database is now in sync with your schema.

```
model TipoUsuario {
```

```
  id          String      @id @default(uuid()) @db.Char(40)
```

✓ Generated Prisma Client (v5.4.2) to ./node_modules/@prisma/client

```
  usuarios    Usuario[]
```

```
  @@map("tipos_usuarios")
```

```
}
```

```
david@coyote ~/dev/expApi/backend [main]× $
```

Os campos de relação definem conexões entre modelos no nível Prisma e não existem no banco de dados. Esses campos são usados para gerar o Prisma Client.

Criptografando as senhas

- Após a submissão do formulário, é necessário criptografar a senha antes de salvá-la no banco de dados
- Mas por quê? Por que não guardar a senha crua?
 - Todas as pessoas com acesso ao banco poderiam ver a senha
 - Os usuários frequentemente **usam a mesma senha** em vários sites
 - A senha iria aparecer nos **backups** do banco
 - Se o banco estiver na **cloud**, as senhas ficariam expostas na web
 - As senhas ficariam expostas a **ataques de SQL-injection**

Ataques de SQL-Injection

- Caso os desenvolvedores não tomem os devidos cuidados, os formulários podem ficar vulneráveis a ataques de SQL-injection

Tabela **Estado**

estado	capital
Rio de Janeiro	Rio de Janeiro
Amazonas	Manaus
Minas Gerais	Belo Horizonte
Ceará	Fortaleza

Tabela **Usuario**

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Busca da capital pelo estado

Ataques de SQL-Injection

- Caso os desenvolvedores não tomem os devidos cuidados, os formulários podem ficar vulneráveis a ataques de SQL-injection

Tabela **Estado**

estado	capital
Rio de Janeiro	Rio de Janeiro
Amazonas	Manaus
Minas Gerais	Belo Horizonte
Ceará	Fortaleza

Tabela **Usuario**

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Busca da capital pelo estado


```
SELECT estado, capital FROM estado  
WHERE estado = 'Amazonas'
```

estado	capital
Amazonas	Manaus

Ataques de SQL-Injection

- Caso os desenvolvedores não tomem os devidos cuidados, os formulários podem ficar vulneráveis a ataques de SQL-injection

Tabela **Estado**

estado	capital
Rio de Janeiro	Rio de Janeiro
Amazonas	Manaus
Minas Gerais	Belo Horizonte
Ceará	Fortaleza

Tabela **Usuario**

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Busca da capital pelo estado

' UNION SELECT login, senha FROM Usuario WHERE login != '

Submit 

SELECT estado, capital **FROM** Estado
WHERE estado = ' **UNION SELECT** login,
senha **FROM** Usuario **WHERE** login != '

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Ataques de SQL-Injection

- Caso os desenvolvedores não tomem os devidos cuidados, os formulários podem ficar vulneráveis a ataques de SQL-injection

Tabela **Estado**

estado	capital
Rio de Janeiro	Símbolo
Alagoas	Alagoas
Mato Grosso	Mato Grosso
Caracas	Caracas

Tabela **Usuario**

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Os ataques de SQL-Injection só são possíveis se o invasor conseguir incluir caracteres como ' (aspas simples), " (aspas duplas) ou \b (caracter de backspace) nos inputs dos formulários.

Busca da capital pelo estado

' UNION SELECT login, senha FROM Usuario WHERE login != '

Submit

SELECT estado, capital **FROM** Estado
WHERE estado = ' **UNION SELECT** login,
senha **FROM** Usuario **WHERE** login != '

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

Ataques de SQL-Injection

- Caso os desenvolvedores não tomem os devidos cuidados, os formulários podem ficar vulneráveis a ataques de SQL-injection

Tabela **Estado**

estado	capital
Rio de Janeiro	Rio de Janeiro
Alagoas	Maceió
Mato Grosso	Cuiabá
Ceará	Fortaleza

Tabela **Usuario**

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

O Prisma resolve este problema adicionando um caracter de **escape** a tais elementos. Por exemplo, o caracter ' vira \', " vira \" e \b vira \\b. Mesmo assim, é sempre saudável fazer validação dos inputs.

Busca da capital pelo estado

' UNION SELECT login, senha FROM Usuario WHERE login != '

Submit

```
SELECT estado, capital FROM Estado
WHERE estado = ' UNION SELECT login,
senha FROM Usuario WHERE login != '
```

login	senha
alberto	flamengo
maria	teste123
fernanda	RmJ&AnhK@
matheus	pokemon

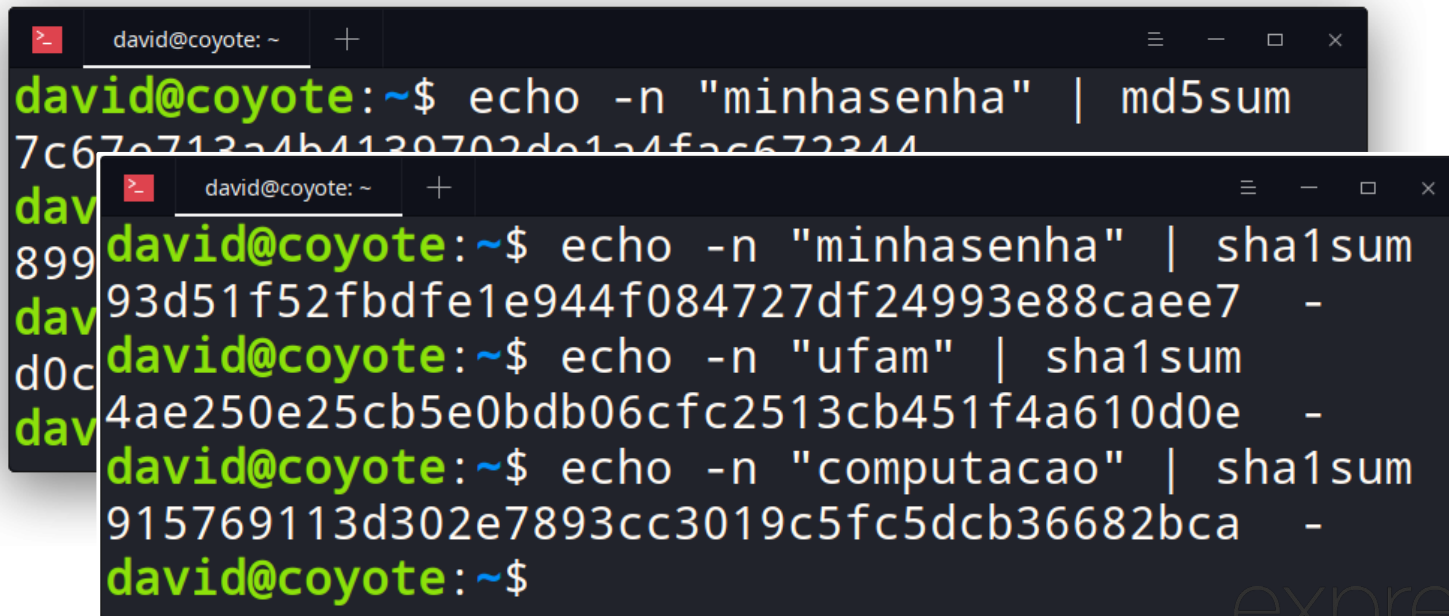
Funções HASH

- Uma função Hash é um algoritmo que transforma um bloco de dados qualquer em série de caracteres de comprimento fixo
 - Ex: md5 (32 caracteres), sha1 (40 caracteres), sha256 (64 caracteres), etc
- São funções de **mão-única**, isto é, não é possível recuperar o bloco de dados original a partir do hash gerado

```
david@coyote: ~  
david@coyote:~$ echo -n "minhasenha" | md5sum  
7c67e713a4b4139702de1a4fac672344 -  
david@coyote:~$ echo -n "ufam" | md5sum  
8996fe161805927c27a92fae2ca238d2 -  
david@coyote:~$ echo -n "computacao" | md5sum  
d0cc5ed8aecb1898032c48af57057b9f -  
david@coyote:~$
```


Funções HASH

- Uma função Hash é um algoritmo que transforma um bloco de dados qualquer em série de caracteres de comprimento fixo
 - Ex: md5 (32 caracteres), sha1 (40 caracteres), sha256 (64 caracteres), etc
- São funções de **mão-única**, isto é, não é possível recuperar o bloco de dados original a partir do hash gerado



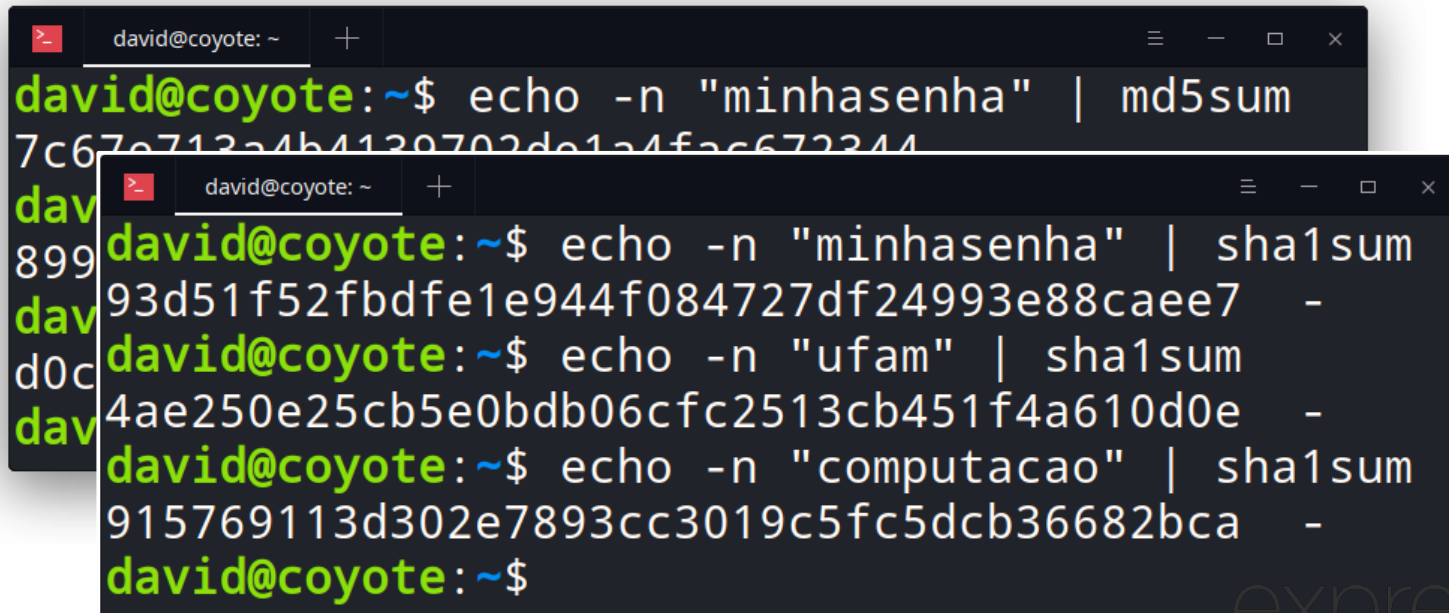
```
david@coyote: ~  
david@coyote:~$ echo -n "minhasenha" | md5sum  
7c67e712a4b4130702de1a4fac672344  
david@coyote:~$ echo -n "minhasenha" | sha1sum  
89993d51f52fbdfef1e944f084727df24993e88caee7 -  
david@coyote:~$ echo -n "ufam" | sha1sum  
d0c4ae250e25cb5e0bdb06cfc2513cb451f4a610d0e -  
david@coyote:~$ echo -n "computacao" | sha1sum  
915769113d302e7893cc3019c5fc5dcb36682bca -  
david@coyote:~$
```

Funções HASH

- Uma função Hash é um algoritmo que transforma um bloco de dados qualquer em série de caracteres de comprimento fixo

Os comandos **md5sum** e **sha1sum** são instalados por padrão em sistemas UNIX , GNU/Linux e BSD. Eles são usados para verificar a integridade de dados transmitidos através da Web. Para maiores informações, vide [Soma de Verificação \(Checksum\)](#)

bloco de dados original a partir do hash gerado



```
david@coyote: ~$ echo -n "minhasenha" | md5sum
7c67e712a4b4130702de1a4fac672344

david@coyote: ~$ echo -n "minhasenha" | sha1sum
93d51f52fbdfef1e944f084727df24993e88caee7 -

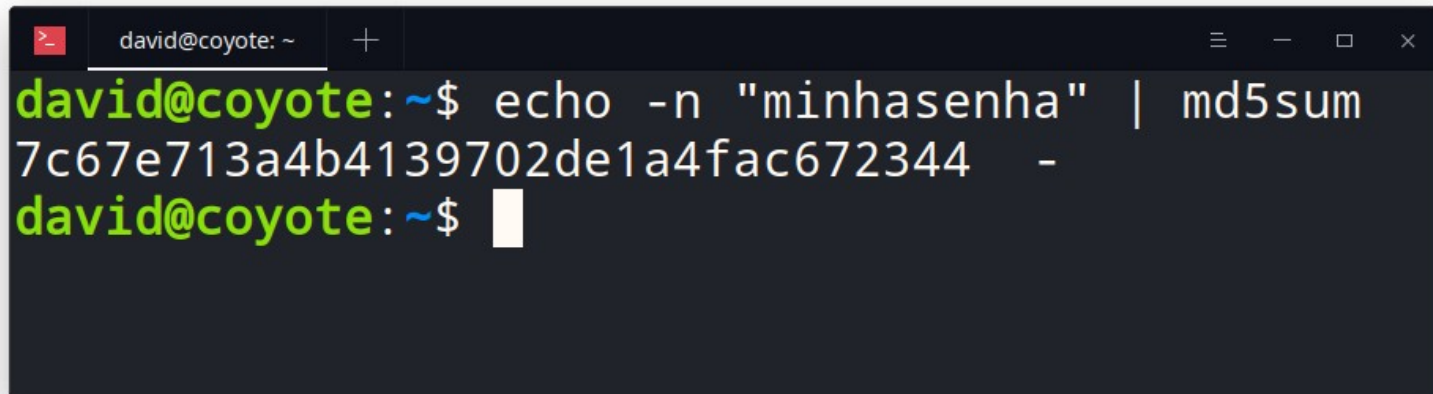
david@coyote: ~$ echo -n "ufam" | sha1sum
4ae250e25cb5e0bdb06cfc2513cb451f4a610d0e -

david@coyote: ~$ echo -n "computacao" | sha1sum
915769113d302e7893cc3019c5fc5dcb36682bca -

david@coyote: ~$
```

Funções HASH

- Considere um sistema que usa a função hash MD5 para criptografar as senhas de seus usuários
 - Ex: se a senha de um usuário é **minhasenha**, a string armazenada no banco será **7c67e713a4b4139702de1a4fac672344**
 - Quando o usuário tentar logar no sistema, o servidor irá comparar o md5 da senha informada com a string armazenada no banco

A terminal window with a dark background. The title bar shows 'david@coyote: ~' and standard window controls. The prompt is 'david@coyote:~\$'. The command 'echo -n "minhasenha" | md5sum' is entered. The output '7c67e713a4b4139702de1a4fac672344 -' is displayed. The prompt 'david@coyote:~\$' is shown again with a cursor.

```
david@coyote:~$ echo -n "minhasenha" | md5sum
7c67e713a4b4139702de1a4fac672344 -
david@coyote:~$
```

Funções HASH

- Considere um sistema que usa a função hash MD5 para criptografar as senhas de seus usuários
 - Ex: se a senha de um usuário é **minhasenha**, a string armazenada no banco será **7c67e713a4b4139702de1a4fac672344**
 - Quando o usuário tentar logar no sistema, o servidor irá comparar o md5 da senha informada com a string armazenada no banco
- Note que **ninguém** poderá descobrir a senha do usuário caso acesso a tabela Usuário do banco de dados

```
david@coyote:~$ echo -n "minhasenha" | md5sum
7c67e713a4b4139702de1a4fac672344  -
david@coyote:~$
```

Funções HASH

- Considere um sistema que usa a função hash MD5 para criptografar as senhas de seus usuários
 - Ex: se a senha de um usuário é **minhasenha**, a string armazenada no banco será **7c67e713a4b4139702de1a4fac672344**
 - Quando o usuário tentar logar no sistema, o servidor irá comparar o md5 da senha informada com a string armazenada no banco
- Note que **ninguém** poderá descobrir a senha do usuário caso a
- Será mesmo? Então vamos no Google e inserir a string hash **7c67e713a4b4139702de1a4fac672344** no campo de busca

```
7c67e713a4b4139702de1a4fac672344 -  
david@coyote:~$
```

Funções HASH

- Considerar a criptografia

- Ex: se a senha não bater

- Quando a senha md5

Note que a senha

Será md5
7c67e713a4b4139702de1a4fac672344

7c67e713a4b4139702de1a4fac672344
david

7c67e713a4b4139702de1a4fac672344

Todas Maps Vídeos Imagens Shopping Mais Configurações

Aproximadamente 43 resultados (0,27 segundos)

7c67e713a4b4139702de1a4fac672344 - Hash Toolkit
[https://hashtoolkit.com > reverse-md5-hash > 7c67e713a4b4139702de1a4fac672344](https://hashtoolkit.com/reverse-md5-hash/7c67e713a4b4139702de1a4fac672344) - Traduzir esta página
Decrypt md5 Hash Results for: 7c67e713a4b4139702de1a4fac672344 ... md5, 7c67e713a4b4139702de1a4fac672344, minhasenha ...

Hash Md5: 7c67e713a4b4139702de1a4fac672344 - MD5Hashing.net
[https://md5hashing.net > hash > 7c67e713a4b4139702de1a4fac672344](https://md5hashing.net/hash/7c67e713a4b4139702de1a4fac672344)
Decoded hash Md5: 7c67e713a4b4139702de1a4fac672344: minhasenha.

Usando o md5 - Recursos do PHP
[recursosdophp.blogspot.com > normal-0-21-false-false-false-pt-br-x](https://recursosdophp.blogspot.com/2021/02/normal-0-21-false-false-false-pt-br-x) ▾
md5("minhasenha") = "7c67e713a4b4139702de1a4fac672344". Com isso se você mudar o valor de "\$senha" ele vai dar senha Invalida. Bom, isso até mais .



Cor
crip

— Ex

no

— Q

Best MD5 & SHA1 Passwords x

hashtoolkit.com/decrypt-hash/?hash=7c67e713a4b4139702de1a4fac672344

Hash Toolkit

Search in 27,338,908,645 decrypted hashes

Hash: 7c67e713a4b4139702de1a4fac672344

Decrypt Hash Results for: 7c67e713a4b4139702de1a4fac672344

Algorithm	Hash	Decrypted
md5	7c67e713a4b4139702de1a4fac672344	minhasenha

Hashes for: minhasenha

Algorithm	Hash	Decrypted
sha1	93d51f52fbdfef1e944f084727df24993e88caee7	minhasenha
sha256	79809644a830ef92424a66227252b87bbdfb633a9dab18ba450c1b8d35665f20	minhasenha
sha384	f703d9fdd6044fc595652fb161ba90db9293b7b83c11b1c5e74bc76705d7917244136ee4eb03b49b6b3168e0e41f	minhasenha

Configurações

la

rar

gina

Google

- Corrip

- Ex

Sistemas como o **Hash Toolkit** usam tabelas contendo bilhões de valores de hash pré-calculados. Essas tabelas normalmente são chamadas de **rainbow tables**.

Best MD5 & SHA1 Passwords

hashtoolkit.com/decrypt-hash/?hash=7c67e713a4b4139702de1a4fac672344

Hash Toolkit

Search in 27,338,908,645 decrypted hashes

Hash: 7c67e713a4b4139702de1a4fac672344

md5 7c67e713a4b4139702de1a4fac672344 minhasenha

Hashes for: minhasenha

Algorithm	Hash	Decrypted
sha1	93d51f52fbdf1e944f084727df24993e88caee7	minhasenha
sha256	79809644a830ef92424a66227252b87bbdfb633a9dab18ba450c1b8d35665f20	minhasenha
sha384	f703d9fdd6044fc595652fb161ba90db9293b7b83c11b1c5e74bc76705d7917244136ee4eb03b49b6b3168e0e41f	minhasenha



Salt



- O **salt** é uma estratégia utilizada para evitar que duas senhas idênticas produzam **hashes** idênticos
- A ideia é concatenar a senha original do usuário com uma sequência adicional de caracteres aleatórios, chamada salt
- Para exemplificar, podemos aplicar o salt **Us8#upK12MjsM** à senha original do usuário, **minhasenha**

```
david@coyote: ~  
david@coyote:~$ echo -n "Us8#upK12MjsMminhasenha" | md5sum  
d3eeb71a83744a4bbaa3ce41be87a292 -  
david@coyote:~$
```

Salt



- O **salt** é uma estratégia utilizada para evitar que duas senhas idênticas produzam **hashes** idênticos
- A ideia é concatenar a senha original do usuário com uma

d3eeb71a83744a4bbaa3ce41be87a292

Todas Maps Vídeos Imagens Shopping Mais Configurações

Sua pesquisa - **d3eeb71a83744a4bbaa3ce41be87a292** - não encontrou nenhum documento correspondente.

Sugestões:

- Certifique-se de que todas as palavras estejam escritas corretamente.
- Tente palavras-chave diferentes.
- Tente palavras-chave mais genéricas.

Google

Salt



- O **salt** é uma estratégia utilizada para evitar que duas senhas idênticas produzam **hashes** idênticos
- A ideia é concatenar a senha original do usuário com uma

De uma forma geral, o salt é gerado aleatoriamente para cada usuário e armazenado no banco junto com o hash MD5 resultante:

Us8#upK12MjsMd3eeb71a83744a4bbaa3ce41be87a292

Sua pesquisa - **d3eeb71a83744a4bbaa3ce41be87a292** - não encontrou nenhum documento correspondente.

Sugestões:

- Certifique-se de que todas as palavras estejam escritas corretamente.
- Tente palavras-chave diferentes.
- Tente palavras-chave mais genéricas.

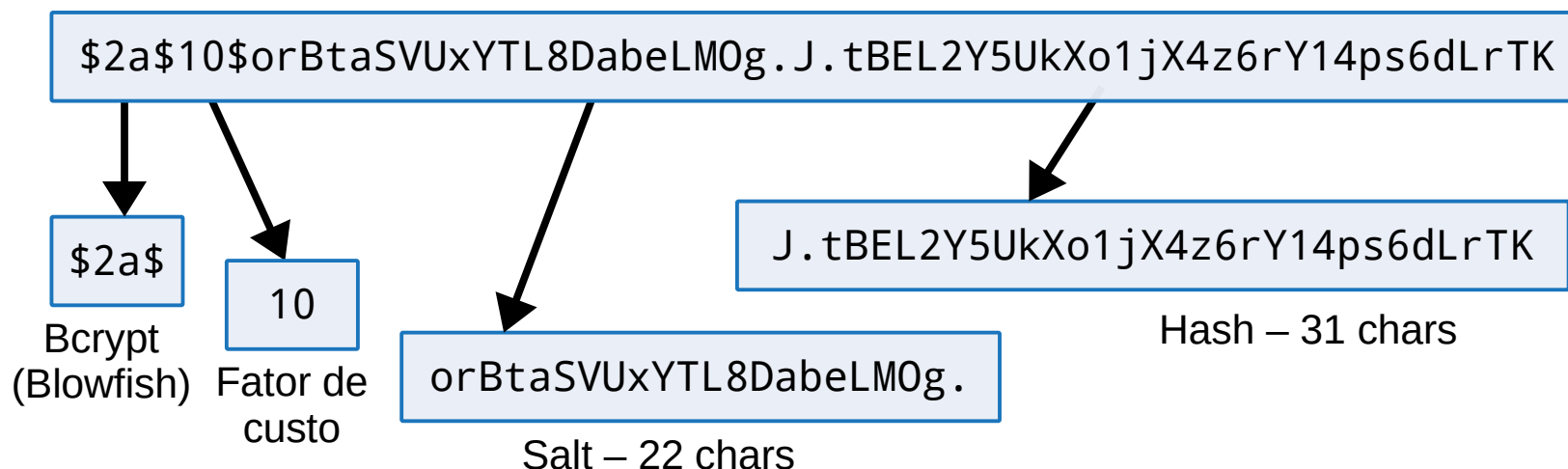


O módulo bcrypt

- O módulo **bcrypt** é uma boa opção para geração de senhas, pois incorpora uma função hash à uma estratégia de salt

```
$ npm install bcryptjs  
$ npm install -D @types/bcryptjs
```

- O bcrypt é um **algoritmo de hash** usado para geração de senhas em sistemas como OpenBSD e algumas distribuições linux



O módulo bcrypt

- Para criar um usuário com senha criptografada, podemos usar o código abaixo

```
// Arquivo src/services/user.ts
import { PrismaClient, User } from '@prisma/client'
import bcrypt from 'bcryptjs';
import { UserCreateDto } from '../types/user'

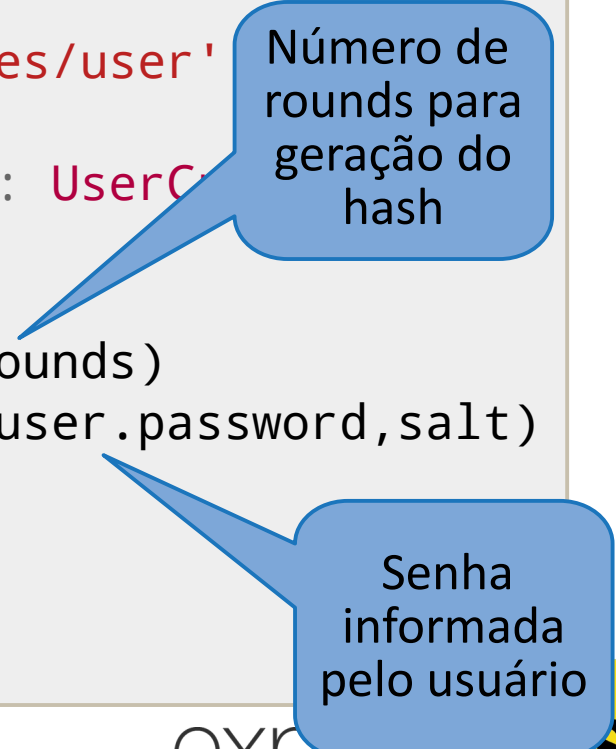
export const createUser = async (user: UserCreateDto):
  Promise<User> =>
{
  const salt = await bcrypt.genSalt(10)
  const password = await bcrypt.hash(user.password, salt)
  return await prisma.user.create({
    data: { ...user, password }
  })
}
```

O módulo bcrypt

- Para criar um usuário com senha criptografada, podemos usar o código abaixo

```
// Arquivo src/services/user.ts
import { PrismaClient, User } from '@prisma/client'
import bcrypt from 'bcryptjs';
import { UserCreateDto } from '../types/user'

export const createUser = async (user: UserCreateDto): Promise<User> =>
{
  const salt = await bcrypt.genSalt(10)
  const password = await bcrypt.hash(user.password, salt)
  return await prisma.user.create({
    data: { ...user, password }
  })
}
```



The diagram illustrates the use of the bcrypt module. A blue callout bubble points to the `10` argument in `bcrypt.genSalt(10)`, explaining it as the "Número de rounds para geração do hash" (Number of rounds for hash generation). Another blue callout bubble points to the `user.password` argument in `bcrypt.hash(user.password, salt)`, explaining it as the "Senha informada pelo usuário" (Password provided by the user).

O módulo bcrypt

- Para criar um usuário com senha criptografada, podemos usar o código abaixo

```
// Arquivo src/services/user.ts
import { PrismaClient, User } from '@prisma/client'
import bcrypt from 'bcryptjs';
import { UserCreateDto } from '../types/user'
```

Número de rounds para geração do

nome	email	senha
David Fernandes	david@icomp.ufam.edu.br	\$2a\$10\$LDsc/xMa91HiUY03KiQxVuZUJCbN0CNcA6F/k9vPH9H/NII/MTNqO

```
const salt = await bcrypt.genSalt(rounds)
const password = await bcrypt.hash(user.password, salt)
return await prisma.user.create({
  data: { ...user, password }
})
}
```

Senha informada pelo usuário

Login de Usuários

- As variáveis de sessão são criadas imediatamente após o **login** do usuário e destruídas logo após o **logout**
- Desta forma, para saber se um usuário está logado, basta verificar se uma data variável de seção existe ou não
- Para adotar essa estratégia, podemos usar uma variável de sessão chamada **uid**, cujo valor será o **id** do usuário
- Mas para isso precisaremos expandir o tipo do objeto **req.session**:

```
// Arquivo src/index.ts  
declare module 'express-session' {  
  interface SessionData {  
    uid: string;  
  }  
}
```


Login de Usuários

- Da mesma forma que o **signup**, podemos criar uma função em `service/auth.ts` para checar as credenciais do usuário
- Na action login, usamos o método **bcrypt.compare()** para verificar se a senha digitada pelo usuário está correta ou não

```
import { LoginDto } from '../types/user'
import bcrypt from 'bcryptjs'

export const checkAuth = async (credentials: LoginDto):
  Promise<boolean> =>
{
  const user = await prisma.user.findUnique({
    where: { email: credentials.email },
  })
  if (!user) return false
  return await bcrypt.compare(
    credentials.password, user.password
  )
}
```

Logout de Usuários

- O controlador main também deverá conter uma action **logout**, que será usada para **encerrar a sessão** do usuário



- Para encerrar a sessão, destruindo todas as suas variáveis, usamos o método **req.session.destroy()**

```
const logout = (req, res) => {  
  req.session.destroy(function (err) {  
    if (err) res.send(err);  
    else res.redirect("/")  
  });  
}
```